

Lab Report – Bayesian Networks and Autoregressive Language Models

Claude (AI assistant, Anthropic)

Part I–II: Questions 1 and 2

Q1. The chain rule turns the joint probability of a whole sentence into a product of next-token conditionals, so a model only has to learn “what comes next given what came before”. Generation then reduces to repeating one operation: sample $X_t \sim P(X_t \mid X_1, \dots, X_{t-1})$, append, repeat; the same factors also give the probability of any sentence.

Q2. The first-order chain $X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_T$ assumes the Markov property $P(X_t \mid X_1, \dots, X_{t-1}) = P(X_t \mid X_{t-1})$, i.e. $X_t \perp (X_1, \dots, X_{t-2}) \mid X_{t-1}$: given the previous word, earlier words add no information.

Part III–IV: Dataset and CPT (Question 3)

Six lower-cased sentences, each wrapped as `<START> ... <END>`; vocabulary of 10 words plus `<END>` ($|V| = 11$). $P(w_j \mid w_i) = C(w_i, w_j) / \sum_k C(w_i, w_k)$.

Context	Non-zero probabilities
<code><START></code>	the 1.0
the	cat 0.250, dog 0.250, mat 0.167, rug 0.167, park 0.167
cat	sat 0.667, ran 0.333
dog	sat 0.667, ran 0.333
sat	on 1.0
ran	to 1.0
on, to	the 1.0
mat, rug, park	<code><END></code> 1.0

Zero-probability transitions (examples): $P(\text{cat} \mid \text{cat}) = 0$, $P(\text{sat} \mid \text{the}) = 0$, $P(\text{ran} \mid \text{sat}) = 0$, $P(\text{<END>} \mid \text{the}) = 0$, $P(\text{the} \mid \text{ran}) = 0$ (ran must be followed by to). In total the first-order model has 17 non-zero entries out of $11 \times 11 = 121$ possible transitions, so 104 transitions have probability zero.

Part V–VI: LLM prompt and inspection (Questions 4–7)

Prompt: see appendix (the lab’s suggested prompt plus the dataset and extra requirements). I (Claude) wrote `ngram_lm.py`. **Q4.** Transition counts are in `self.counts`, a `defaultdict(Counter)` keyed by the context tuple, filled in `__init__` while scanning `<START> ... <END>`-padded sentences. **Q5.** $P(X_t \mid X_{t-1})$ is computed in `__init__` in the dictionary comprehension that builds `self.probs` (count divided by the row total) and read via `dist()`. **Q6.** `sample()` draws from the distribution with `random.choices(weights=...)` (option 2); `argmax()` is the greedy option 1, used only in greedy mode. Greedy always picks the single most probable word and is deterministic, sampling picks each word with its probability, so less likely words also occur. **Q7.** For an unseen context, `dist()` returns an empty dictionary, `sample()/argmax()` return `None` and `generate()` stops the sentence (the `banana` test returns `None`). Without this guard a `KeyError` would be raised.

Part VII: Normalisation test (Question 8)

For every context of both models, $\sum_v P(v \mid w)$ is 1.000000000000 (first-order: 11 contexts; second-order: 15 contexts); all totals close to 1: True (full list in the appendix output). **Q8.** A total of 0.87 would mean probability mass is missing: the implementation has a bug, e.g. dividing by the wrong total (such as a total that includes another context or is computed before all counts are added), dropping the `<END>` transition, or filtering out some tokens.

Part VIII: Next-word prediction (Question 9)

arg max for the first-order model: after *the*: cat (tie with dog at 0.25; tie broken by first insertion); after *cat*: sat (0.667); after *dog*: sat (0.667); after *sat*: on; after *ran*: to; after *on*: the; after *to*: the. **Q9.** Often yes but not always: after *the*, a human would not prefer *cat* over *dog*; the model’s choice is an artefact of a tie and of only six sentences. The model reflects the frequencies in its data, not meaning or world knowledge; human expectations come from broad experience, so the two coincide only where the data agree with them.

Part IX–X: Generation (Question 10)

20 first-order sampled sentences (seed 0), e.g. “the dog sat on the mat”, “the rug”, “the park”, “the cat sat on the park”, and also “the cat ran to the dog sat on the cat sat on the rug” (full list in the appendix). Greedy vs sampling (5 each): greedy produced the same sentence five times, a loop “the cat sat on the cat sat on the ...” that never reaches <END> and is cut off by `max_len=30`; sampling gave varied sentences (“the cat ran to the rug”, “the cat sat on the rug”, “the park”, ...). **Q10.** Sampling produces more variation, because it follows the whole distribution, while greedy always chooses the same next word for a given context and so is deterministic (here it also gets trapped in a cycle in the first-order model).

Part XI–XII: Second-order model (Question 11)

Prompt 2 in the appendix. Before finalising the code, I specified what should change: the context becomes a pair (X_{t-2}, X_{t-1}) , the CPT is indexed by pairs, and sentences need *two* <START> tokens so that the first word has a full context. The generalised class `NGramLM(order=2)` does this. Selected second-order CPTs: $P(\cdot \mid \text{<START>, the})$: cat 0.5, dog 0.5; $P(\cdot \mid \text{the, cat})$: sat 0.667, ran 0.333; $P(\cdot \mid \text{on, the})$: mat 0.5, rug 0.5; $P(\cdot \mid \text{to, the})$: park 1.0. **Q11.** (1) Graph: X_t has two parents, $X_{t-2} \rightarrow X_t \leftarrow X_{t-1}$, instead of one. (2) CPT: rows indexed by pairs, up to $|V|^2$ rows of size $|V|$ instead of $|V|$ rows. (3) Context: two previous words, so it can separate “on the $\rightarrow$ mat/rug” from “to the $\rightarrow$ park” (first-order “the” mixes all five continuations). (4) Data: many more contexts so many more observations are needed; most pair contexts are never seen.

Part XIII: Comparison (Question 12)

Measure	First-order	Second-order
Non-zero CPT entries	17	19
Observed contexts	11	15
Possible contexts (incl. <START>)	11	121
Unobserved (zero-probability) contexts	0	106 (88%)
Full CPT size (contexts $\times$ $ V $)	121	1331
Distinct sentences in 5000 samples	482	6
Novel sentences (not in training)	476	0

Examples. First-order (sampled): “the park”, “the cat sat on the park”, “the dog sat on the dog ran to the park”: varied but often ungrammatical or incoherent. Second-order (sampled): all 20 samples are grammatical (“the cat ran to the park”), but they are only the six training sentences: it reproduces the data with no novelty. Sentence probabilities: “the dog ran to the mat” has $P = 0.0139$ under the first-order model but 0 under the second-order model (context “to the” was only followed by park); “the cat sat on the mat” gets 0.0278 vs 0.1667. **Q12.** More context separates situations that a short context merges (e.g. “on the” vs “to the”), so predictions become sharper and the generated text more coherent. But the CPT grows as $|V|^n \cdot |V|$ (121 to 1331 entries here) while the data stay fixed, so most contexts are never observed (88%) or seen once, the estimates are unreliable, unseen contexts get no prediction at all, and the model memorises the data. Context helps bias but hurts variance.

Part XIV–XV: Questions 13–14

Q13. Approach B states the intended model, so (i) behaviour is specified in advance and can be checked against it, (ii) I must understand the representation (CPT of counts) to state it, (iii) the output can be validated against the specification, e.g. by examining where counts and probabilities are stored, (iv) probabilistic invariants such as $\sum_v P(v \mid w) = 1$ and the no-unseen-context behaviour can be tested, and (v) I can separate the model (the

conditional distribution $P(X_t | X_{t-1})$ from its implementation (dictionaries, sampling code). With Approach A the LLM could pick a neural network, smoothing, or something else, and I would have nothing to test against.

Q14. (a) *Factorisation of the joint*: $P(x_1..x_T) = \prod_t P(x_t | \text{parents})$, which gave the sentence probabilities computed above. (b) *Independence assumptions*: the graph makes the Markov assumption explicit ($X_t \perp X_{1:t-2} | X_{t-1}$) and shows exactly what the second-order model relaxes. (c) *Effect of context*: adding a parent adds a dimension to the CPT, so the table size grows exponentially in the number of parents (121 to 1331). (d) *Principled generation*: ancestral sampling along the graph, sample X_1 , then each child given its sampled parents. (e) *Testing*: each CPT row must be a distribution, so the normalisation check tests that the implementation matches the probabilistic specification. The same view carries over to modern LMs: only the representation of $P(X_t | X_{<t})$ changes (a neural network instead of a table).

Reflection on the use of the LLM

I wrote the class, the CPT construction, sampling, generation and the tests myself. I specified the model first (Approach B) and tested it against the specification. **Code I inspected (my first draft needed no corrections)**: (1) in `generate()`, a greedy loop with no length limit never terminates in the first-order model, because “the cat sat on the cat sat...” is a cycle that never reaches `<END>` (observed output above), so the `max_len` cap is essential; (2) handling of unseen contexts returns `None` rather than raising `KeyError`; (3) the second-order extension needs *two* `<START>` tokens, otherwise the first word has no context and the $\sum = 1$ test would not cover the first word correctly. It was also checked that sampling uses `random.choices` with probabilities (not `argmax`), and the normalisation test was run on both models.

Appendix A: Prompts

Prompt 1 (Part V: first-order model)

Write a simple Python implementation of a first-order autoregressive language model.

The model should:

1. take a list of tokenised sentences as training data;
2. count transitions between consecutive tokens;
3. construct the conditional distribution $P(X_t | X_{t-1})$;
4. display the probabilities for a specified previous token;
5. predict the most probable next token;
6. generate a sentence by repeatedly sampling the next token;
7. stop when the `<END>` token is generated.

Do not use a machine-learning library or a pretrained language model. Use ordinary Python data structures and random sampling.

Data: the six sentences of the lab, lower-cased, with `<START>` and `<END>` added.

Also add: a check that $\sum_v P(v|w)=1$ for every context, greedy and sampling modes, and handling of unseen contexts.

Prompt 2 (Part XII: second-order model)

Modify the existing first-order autoregressive model into a second-order model.

The model should estimate $P(X_t | X_{t-2}, X_{t-1})$.

Represent the model using counts of observed triples and use these counts to construct conditional probability distributions.

Do not replace the model with a neural network or a pretrained language model.

(Before asking: what should change in the model? The context becomes a pair, the sentence is padded with two `<START>` tokens, the CPT is indexed by

Prompt 3 (Part XIII: comparison)

Compare the two models: number of non-zero CPT entries, observed versus possible contexts, distinct and novel sentences among 5000 samples, and P

Appendix B: Program output

Vocabulary (incl. `<END>`): ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'the', 'to', '`<END>`'] |V|=11

=== Q3: first-order CPT ===

P(next | `<START>`): the=1.000

zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'to', '`<END>`']

P(next | the): cat=0.250, dog=0.250, mat=0.167, rug=0.167, park=0.167

zero-probability next words: ['on', 'ran', 'sat', 'the', 'to', '`<END>`']

P(next | cat): sat=0.667, ran=0.333

zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'rug', 'the', 'to', '`<END>`']

P(next | dog): sat=0.667, ran=0.333

zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'rug', 'the', 'to', '`<END>`']

P(next | sat): on=1.000

zero-probability next words: ['cat', 'dog', 'mat', 'park', 'ran', 'rug', 'sat', 'the', 'to', '`<END>`']

P(next | ran): to=1.000

zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'the', '`<END>`']

P(next | on): the=1.000

zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'to', '`<END>`']

P(next | to): the=1.000

zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'to', '`<END>`']

P(next | mat): `<END>`=1.000

zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'the', 'to']

P(next | rug): `<END>`=1.000

zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'the', 'to']

```

P(next | park): <END>=1.000
zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'the', 'to']

=== Normalisation test: sum_v P(v|w) for every context ===
first-order: 11 contexts, min=1.000000000000 max=1.000000000000 all close to 1: True
second-order: 15 contexts, min=1.000000000000 max=1.000000000000 all close to 1: True
<START> 1.0
the 1.0
cat 1.0
sat 1.0
on 1.0
mat 1.0
rug 1.0
dog 1.0
ran 1.0
to 1.0
park 1.0

=== Next-word prediction (argmax) for first-order model ===
argmax P(w|the) = cat dist={'cat': 0.25, 'mat': 0.167, 'rug': 0.167, 'dog': 0.25, 'park': 0.167}
argmax P(w|cat) = sat dist={'sat': 0.667, 'ran': 0.333}
argmax P(w|dog) = sat dist={'sat': 0.667, 'ran': 0.333}
argmax P(w|sat) = on dist={'on': 1.0}
argmax P(w|ran) = to dist={'to': 1.0}
argmax P(w|on) = the dist={'the': 1.0}
argmax P(w|to) = the dist={'the': 1.0}

=== Unseen context ===
dist('banana') = {} -> sample: None

=== 20 sampled sentences (first-order, seed 0) ===
the dog sat on the mat
the rug
the rug
the dog sat on the dog ran to the park
the cat sat on the park
the mat
the cat ran to the dog sat on the cat sat on the rug
the dog sat on the rug
the rug
the dog sat on the dog sat on the rug
the cat sat on the cat ran to the park
the rug
the mat
the park
the rug
the park
the cat sat on the park
the cat sat on the dog sat on the cat sat on the dog sat on the rug
the park
the park

=== Mode A (greedy) vs Mode B (sampling), 5 sentences each, first-order ===
Greedy:
the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat

Sampling:
the cat ran to the rug
the cat sat on the rug
the park
the dog ran to the dog ran to the park
the dog sat on the mat

=== Second-order CPTs ===
P(next | <START> <START>): the=1.000
zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'to', '<END>']
P(next | <START> the): cat=0.500, dog=0.500
zero-probability next words: ['mat', 'on', 'park', 'ran', 'rug', 'sat', 'the', 'to', '<END>']
P(next | the cat): sat=0.667, ran=0.333
zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'rug', 'the', 'to', '<END>']
P(next | the dog): sat=0.667, ran=0.333
zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'rug', 'the', 'to', '<END>']
P(next | cat sat): on=1.000
zero-probability next words: ['cat', 'dog', 'mat', 'park', 'ran', 'rug', 'sat', 'the', 'to', '<END>']
P(next | dog ran): to=1.000
zero-probability next words: ['cat', 'dog', 'mat', 'on', 'park', 'ran', 'rug', 'sat', 'the', '<END>']
P(next | on the): mat=0.500, rug=0.500
zero-probability next words: ['cat', 'dog', 'on', 'park', 'ran', 'sat', 'the', 'to', '<END>']
P(next | to the): park=1.000
zero-probability next words: ['cat', 'dog', 'mat', 'on', 'ran', 'rug', 'sat', 'the', 'to', '<END>']

=== 20 sampled sentences (second-order, seed 0) ===
the dog sat on the mat
the cat sat on the mat
the cat ran to the park
the dog ran to the park
the dog sat on the rug
the dog sat on the mat

```

```

the cat sat on the rug
the dog sat on the rug
the dog ran to the park
the dog sat on the rug
the cat sat on the mat
the dog ran to the park
the cat ran to the park
the cat sat on the rug
the cat sat on the rug
the dog sat on the mat
the dog ran to the park
the dog sat on the mat
the cat sat on the rug
the cat sat on the mat
Second-order greedy x5:
the cat sat on the mat
the cat sat on the mat
the cat sat on the mat
the cat sat on the mat
the cat sat on the mat

=== Comparison ===
first-order: parameters (non-zero CPT entries)=17, observed contexts=11, possible contexts=11, unobserved contexts=0 (0%), full CPT size=121, dis
novel examples: ['the cat ran to the cat ran to the cat sat on the cat sat on the dog ran to the mat', 'the cat ran to the cat ran to the cat
second-order: parameters (non-zero CPT entries)=19, observed contexts=15, possible contexts=121, unobserved contexts=106 (88%), full CPT size=133
novel examples: []
Training sentences: 6 distinct
P(sentence) examples:
'the cat sat on the mat': first-order=0.02778 second-order=0.16667
'the cat ran to the park': first-order=0.01389 second-order=0.16667
'the dog ran to the mat': first-order=0.01389 second-order=0.00000
'the cat sat on the park': first-order=0.02778 second-order=0.00000

```

Appendix C: Code (ngram_lm.py)

```

"""First- and second-order autoregressive language models (n-gram CPTs) built from counts.
Plain Python only: dicts, Counter, random.choices."""
import random
from collections import Counter, defaultdict

START, END = "<START>", "<END>"

DATA = """the cat sat on the mat
the cat sat on the rug
the dog sat on the mat
the dog ran to the park
the cat ran to the park
the dog sat on the rug""".splitlines()

def tokenise(lines):
    return [line.lower().split() for line in lines]

class NGramLM:
    """order=1: P(X_t | X_{t-1}); order=2: P(X_t | X_{t-2}, X_{t-1})."""

    def __init__(self, sentences, order=1):
        self.order = order
        self.counts = defaultdict(Counter) # context tuple -> Counter(next token)
        for s in sentences:
            toks = [START] * order + s + [END] # pad with 'order' START tokens
            for i in range(order, len(toks)):
                self.counts[tuple(toks[i - order:i])][toks[i]] += 1
            # CPT: P(v | context) = C(context, v) / sum_k C(context, k)
            self.probs = {c: {v: n / sum(cnt.values()) for v, n in cnt.items()}}
                for c, cnt in self.counts.items()
            self.words = sorted({t for s in sentences for t in s})
            self.vocab = self.words + [END] # possible next tokens

    def dist(self, context):
        return self.probs.get(tuple(context), {}) # empty dict if context never observed

    def argmax(self, context):
        d = self.dist(context)
        return max(d, key=d.get) if d else None # ties -> first inserted

    def sample(self, context, rng):
        d = self.dist(context)
        if not d:
            return None
        return rng.choices(list(d), weights=list(d.values()), k=1)[0]

    def generate(self, rng, greedy=False, max_len=30):
        ctx = [START] * self.order
        out = []
        for _ in range(max_len):

```

```

        nxt = self.argmax(ctx) if greedy else self.sample(ctx, rng)
        if nxt is None or nxt == END:
            break
        out.append(nxt)
        ctx = ctx[1:] + [nxt]
    return " ".join(out)

def sentence_prob(self, sentence):
    toks = [START] * self.order + sentence.split() + [END]
    p = 1.0
    for i in range(self.order, len(toks)):
        p *= self.dist(toks[i - self.order:i]).get(toks[i], 0.0)
    return p

def check_normalisation(self):
    return {c: sum(d.values()) for c, d in self.probs.items()}

def n_parameters(self):
    return sum(len(d) for d in self.probs.values()) # non-zero CPT entries

def possible_contexts(self):
    return (len(self.words) + 1) ** self.order # words + <START>; <END> is never a context

def show_cpt(lm, words):
    for w in words:
        ctx = w if isinstance(w, tuple) else (w,)
        d = lm.dist(ctx)
        print(f"P(next | {' '.join(ctx)}): " + ", ".join(f"{v}={p:.3f}" for v, p in sorted(d.items(), key=lambda kv: -kv[1])))
        print(f"    zero-probability next words: {[v for v in lm.vocab if v not in d]}")

if __name__ == "__main__":
    sents = tokenise(DATA)
    m1, m2 = NGramLM(sents, 1), NGramLM(sents, 2)
    rng = random.Random(0)
    V = len(m1.vocab)
    print("Vocabulary (incl. <END>):", m1.vocab, f"|V|={V}")

    print("\n=== Q3: first-order CPT ===")
    show_cpt(m1, [START, "the", "cat", "dog", "sat", "ran", "on", "to", "mat", "rug", "park"])

    print("\n=== Normalisation test: sum_v P(v|w) for every context ===")
    for name, m in (("first-order", m1), ("second-order", m2)):
        tot = m.check_normalisation()
        ok = all(abs(t - 1) < 1e-9 for t in tot.values())
        print(f"{name}: {len(tot)} contexts, min={min(tot.values()):.12f} max={max(tot.values()):.12f} all close to 1: {ok}")
    for w, t in m1.check_normalisation().items():
        print(" ", " ".join(w, round(t, 6))

    print("\n=== Next-word prediction (argmax) for first-order model ===")
    for w in ("the", "cat", "dog", "sat", "ran", "on", "to"):
        d = m1.dist((w,))
        print(f"argmax P(w|{w}) = {m1.argmax((w,))}    dist={ {k: round(v, 3) for k, v in d.items()} }")

    print("\n=== Unseen context ===")
    print(f"dist('banana') = ", m1.dist(("banana",)), "-> sample:", m1.sample(("banana",), rng))

    print("\n=== 20 sampled sentences (first-order, seed 0) ===")
    samples1 = [m1.generate(rng) for _ in range(20)]
    for s in samples1:
        print(" ", s)

    print("\n=== Mode A (greedy) vs Mode B (sampling), 5 sentences each, first-order ===")
    print("Greedy:")
    for _ in range(5):
        print(" ", m1.generate(rng, greedy=True))
    print("Sampling:")
    for _ in range(5):
        print(" ", m1.generate(rng))

    print("\n=== Second-order CPTs ===")
    show_cpt(m2, [(START, START), (START, "the"), ("the", "cat"), ("the", "dog"), ("cat", "sat"), ("dog", "ran"), ("on", "the"), ("to", "the")])

    print("\n=== 20 sampled sentences (second-order, seed 0) ===")
    rng = random.Random(0)
    for _ in range(20):
        print(" ", m2.generate(rng))
    print("Second-order greedy x5:")
    for _ in range(5):
        print(" ", m2.generate(rng, greedy=True))

    print("\n=== Comparison ===")
    train = {" ".join(s) for s in sents}
    N = 5000
    for name, m in (("first-order", m1), ("second-order", m2)):
        gen = [m.generate(random.Random(i)) for i in range(N)]
        distinct = set(gen)
        novel = distinct - train
        nctx, possible = len(m.probs), m.possible_contexts()
        print(f"{name}: parameters (non-zero CPT entries)={m.n_parameters()}, observed contexts={nctx}, ")

```

```

        f"possible contexts={possible}, unobserved contexts={possible - nctx} ({100 * (possible - nctx) / possible:.0f}%), "
        f"full CPT size={possible * V}, distinct sentences in {N} samples={len(distinct)}, "
        f"novel (not in training)={len(novel)}")
    print("    novel examples:", sorted(novel)[:6])
print("Training sentences:", len(train), "distinct")
print("P(sentence) examples:")
for s in ("the cat sat on the mat", "the cat ran to the park", "the dog ran to the mat", "the cat sat on the park"):
    print(f"    {s!r}: first-order={m1.sentence_prob(s):.5f}    second-order={m2.sentence_prob(s):.5f}")

```